Crypt-X16

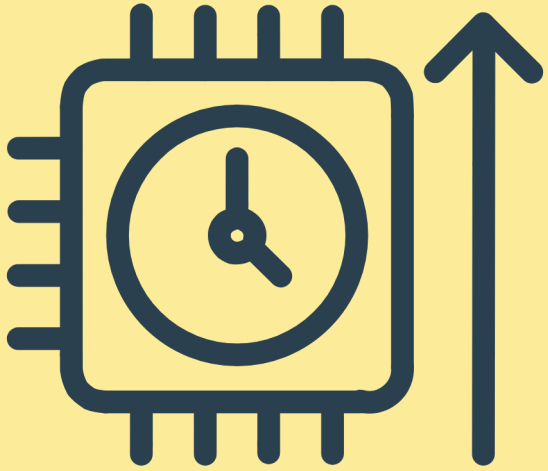
Cryptographic Co-processor using VHDL

VHDL | Hardware

Built in hardware, because software was never fast enough

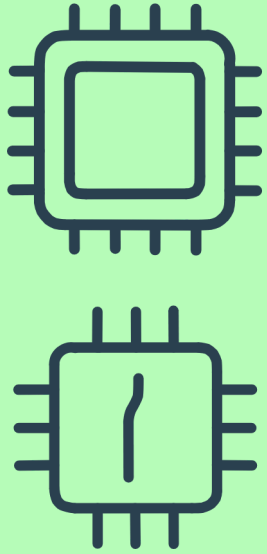
Why a Coprocessor?

Real encryption like AES runs the same operations millions of times per second. Software is too slow.



CPU Alone

Handles everything in software.



CPU + Crypt-X16

Offloads heavy encryption work.

Our coprocessor handles XOR, rotation, substitution and arithmetic in dedicated hardware.

So, What Is Crypt-X16?

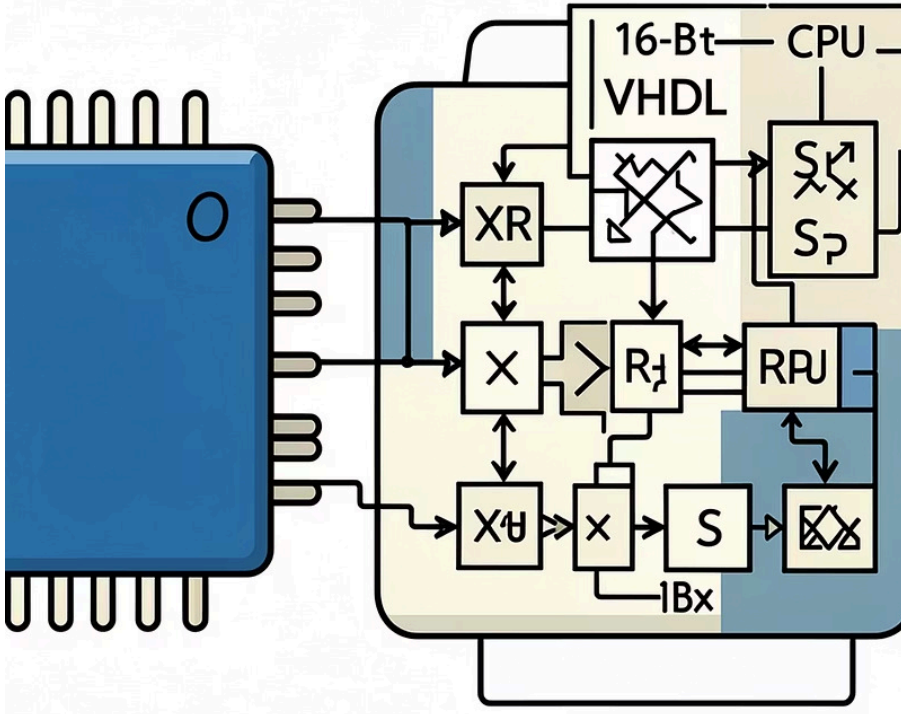
A dedicated hardware unit that runs crypto math so the CPU does not have to.

16-bit VHDL Coprocessor

Sits next to the CPU and handles specific operations in hardware.

Built for Encryption

Implements the exact building blocks AES and similar algorithms rely on.



XOR · ADD · Bit Rotation · S-Box Substitution

System Architecture

The co-processor has three core blocks.

1

I/O Ports

Inputs: Ra, Rb, Rd, ctrl, clk, reset

Outputs: None

2

Input Register

Latches the Ctrl and destination register on every clock edge.

3

16 × 16 Register File

Stores values to be operated on and their results.

4

Combinational Logic Unit

Instantly computes the result using the ALU, shifter, or S-Box depending on Ctrl.

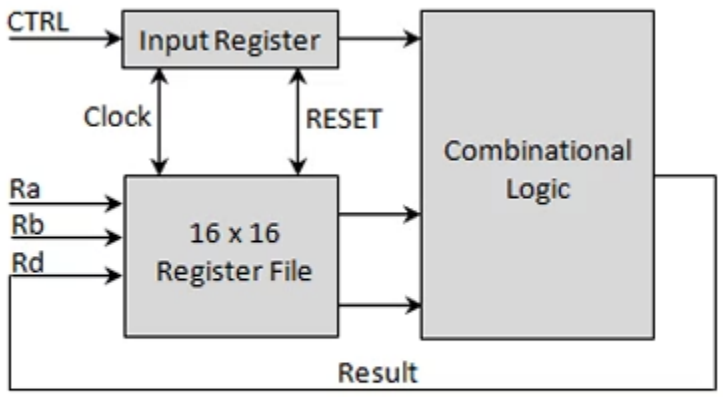


Figure 1. Architecture of the cryptographic coprocessor

One Instruction, Step by Step

1. **Signals Input**
   CTRL=0000, Ra=0101, Rb=0100, Rd=1100
2. **CTRL and Rd Latch**
   The Input Register captures the opcode and destination on the rising clock edge.
3. **Registers Read**
   Register File reads R5=0xF407 onto ABUS and R4=0x1186 onto BBUS
4. **ALU Computes**
   0xF407 + 0x1186 = 0x058D. MUX selects ALU output.
5. **Result Ready**
   0x058D is live on the output wire.
6. **Written Back**
   Next clock edge writes 0x058D into R12.

Ra and Rb are instant. CTRL and Rd wait one clock cycle.

Instruction Set (Ctrl + OP)

ALU Operations

CTRL	Operation	Function
0000	ADD	ABUS + BBUS
0001	SUB	ABUS - BBUS
0010	AND	ABUS & BBUS
0011	OR	ABUS | BBUS
0100	XOR	ABUS ^ BBUS
0101	NOT	~ABUS
0110	MOV	ABUS → RESULT
0111	NOP	Nothing

Shifter & LUT Operations

CTRL	Operation	Function
1000	ROR8	Rotate BBUS right 8 bits
1001	ROR4	Rotate BBUS right 4 bits
1010	SLL8	Shift BBUS left 8, fill zeros
1011	LUT	S-Box substitution

How We Built It

We treated it like a real project. Using **GitHub** Forks, pull requests, code review, nothing touched main without approval.

Our Workflow

Forked the repo

Isolated copies, no breaking the team's work.

One component, one owner

Clear responsibility, no overlap.

PR before merge

Every change reviewed before main.

Lead reviewed everything

Abdallh approved all PRs.

File Structure

```
Design0/src/
├── Components/
│   ├── ALU.vhd
│   ├── shifter.vhd
│   ├── non_linear_lookup.vhd
│   ├── register_file.vhd
│   ├── Control_Logic.vhd
│   ├── MUX.vhd
│   ├── CLU.vhd
│   ├── input_reg.vhd
│   └── crypto_coprocessor.vhd
├── TestBenches/
│   └── one TB per component
└── Schematics/
    └── Active-HDL block diagrams
```

Inside The Crypt-X16

- *CLU (Combinational Logic Unit)*
- *Register*
- *Input Register*
- *TestBenches*

VHDL

HARDWARE DESCRIPTION LANGUAGE

1. Input Register

The pipeline stage that keeps the instruction stable while it executes.

Latches CTRL and Rd

Captures the opcode and destination address on every rising clock edge.

Generates RdWEn

If CTRL is NOP the write enable is forced to 0 and nothing gets saved. On reset it forces CTRL to NOP and clears the write enable.

CTRL and Rd are delayed by exactly one clock cycle. This is the pipeline.

VHDL Modeling

```
if rising_edge(clock) then
   if reset = '1' then
      CTRL_OUT  <= '0111';
      RdWEn_reg <= '0';
   else
      CTRL_OUT <= CTRL_IN;
      Rd_OUT   <= Rd_IN;
      if CTRL_IN = '0111' then
         RdWEn_reg <= '0';
      else
         RdWEn_reg <= '1';
      end if;
   end if;
end if;
```

2. Register File (16 × 16)

The memory of Crypt-X16. Stores all operand values and results.

How It Works

16 registers, 16 bits each

holds all working values for the coprocessor.

Async read

Ra and Rb outputs update instantly with no clock needed.

Sync write

result only writes to Rd on a rising clock edge when RdWEn is 1. NOP disables writing entirely.

On reset, all registers restore to initial values

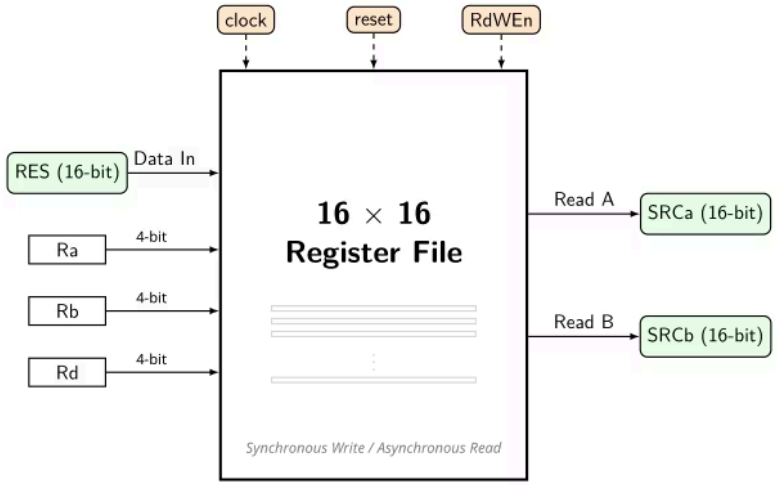


Figure 5: Internal Architecture of the 16×16 Register File Unit.

3. Combinational Logic Unit (CLU)

The heart of Crypt-X16 - three specialized processing units working in parallel

ALU

- ADD, SUB operations
- AND, OR, XOR, NOT logic
- MOV and NOP instructions

Bit Shifter

- ROR8 (rotate right 8 bits)
- ROR4 (rotate right 4 bits)
- SLL8 (shift left 8 bits)

S-Box Lookup

- Nonlinear substitution
- Cryptographic transformation
- AES S-Box implementation

All three units process ABUS and BBUS inputs in parallel. MUX selects the result based on CTRL opcode.

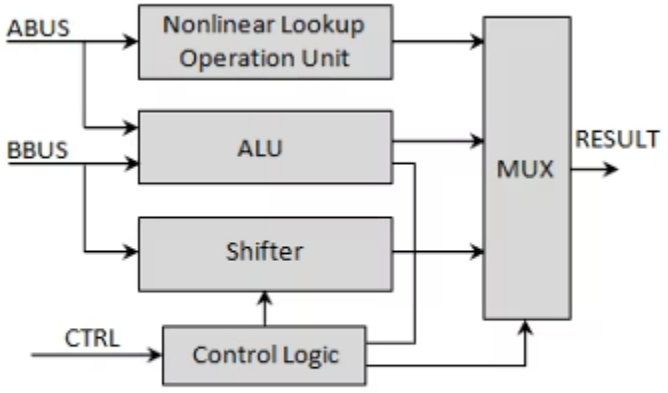


Figure 2. Architecture of the combinational logic block

Control Logic & MUX

The control logic interprets the `CTRL` signal to select which functional unit (ALU, Shifter, or S-Box) processes the data and routes its output through the Multiplexer (MUX) to the result bus.

Control Flow

ALU Operations

When `CTRL(3)` is '0', the lower 3 bits select the ALU operation.

Shifter Operations

For `CTRL` codes 1000-1010, the Shifter is enabled.

S-Box Lookup

When `CTRL` is 1011, S-Box performs substitution.

MUX Selection

Routes the selected output to the result bus based on `MUX_SEL`.

VHDL Modeling

```
if CTRL(3) = '0' then
   ALU_Ctrl <= CTRL(2 downto 0);
   MUX_SEL  <= "01"; -- Route ALU to Result
else
   case CTRL is
      when "1000" =>
         SHIFT_Ctrl <= "00";
         MUX_SEL    <= "10";
      when "1001" =>
         SHIFT_Ctrl <= "01";
         MUX_SEL    <= "10";
      when "1010" =>
         SHIFT_Ctrl <= "10";
         MUX_SEL    <= "10";
      when "1011" =>
         MUX_SEL    <= "11";
      when others =>
         MUX_SEL    <= "00";
   end case;
end if;
```

Arithmetic Logic Unit (ALU)

ALU Operations

ADD

ABUS + BBUS

SUB

ABUS − BBUS

AND

ABUS & BBUS

OR

ABUS | BBUS

Other Operations

XOR

ABUS ^ BBUS

NOT

~ABUS

MOV

ABUS → RESULT

NOP

No operation

VHDL Modeling

```
with ALUctrl select
 ALUOUT <= std_logic_vector(signed(ABUS) + signed(BBUS)) when "000",
        std_logic_vector(signed(ABUS) - signed(BBUS)) when "001",
        ABUS AND BBUS when "010",
        ABUS OR BBUS when "011",
        ABUS XOR BBUS when "100",
        NOT ABUS when "101",
        ABUS when "110",
        (others => '0') when others;
```

Shifter Operations

Shifter Operations

ROR8

The two bytes swap places. `RESULT <= SHIFTINPUT(7 downto 0) & SHIFTINPUT(15 downto 8)`

ROR4

Bottom nibble wraps to the top. `RESULT <= SHIFTINPUT(3 downto 0) & SHIFTINPUT(15 downto 4)`

SLL8

Lower byte moves up, upper byte is lost, zeros fill in. `RESULT <= SHIFTINPUT(7 downto 0) & '00000000'`

VHDL Modeling

```
case SHIFT_Ctrl is
when "00" =>
   SHIFTOUT <= std_logic_vector(rotate_right(unsigned(SHIFTINPUT), 8));
when "01" =>
   SHIFTOUT <= std_logic_vector(rotate_right(unsigned(SHIFTINPUT), 4));
when "10" =>
   SHIFTOUT <= std_logic_vector(shift_left(unsigned(SHIFTINPUT), 8));
when others =>
   SHIFTOUT <= (others => '0');
```

S-Box Nonlinear Lookup

S-Box Operations

Substitution

Nonlinear transformation using lookup table. Maps 8-bit input to 8-bit output.

Input	Output	Input	Output
“0000”	“0001”	“0000”	“1111”
“0001”	“1011”	“0001”	“0000”
“0010”	“1001”	“0010”	“1101”
“0011”	“1100”	“0011”	“0111”
“0100”	“1101”	“0100”	“1011”
“0101”	“0110”	“0101”	“1110”
“0110”	“1111”	“0110”	“0101”
“0111”	“0011”	“0111”	“1010”
“1000”	“1110”	“1000”	“1001”
“1001”	“1000”	“1001”	“0010”
“1010”	“0111”	“1010”	“1100”
“1011”	“0100”	“1011”	“0001”
“1100”	“1010”	“1100”	“0011”
“1101”	“0010”	“1101”	“0100”
“1110”	“0101”	“1110”	“1000”
“1111”	“0000”	“1111”	“0110”

VHDL Modeling

```
-- Step 1: Split input into upper and lower bytes
upper_byte <= LUTIN(15 downto 8); -- pass-through
lower_byte <= LUTIN(7 downto 0);  -- will go through LUT

-- Step 2: Split lower byte into two nibbles
MSN_in <= lower_byte(7 downto 4); -- upper nibble
LSN_in <= lower_byte(3 downto 0); -- lower nibble
```

Our Team

Coding

Abdallh Ashraf (LEADER)

Testbenches, IR, and Co-proccessor

Shaza Walid

CLU and Register File

Osman Mohamed

Control Logic and ALU

Documentation

Ali Khalil

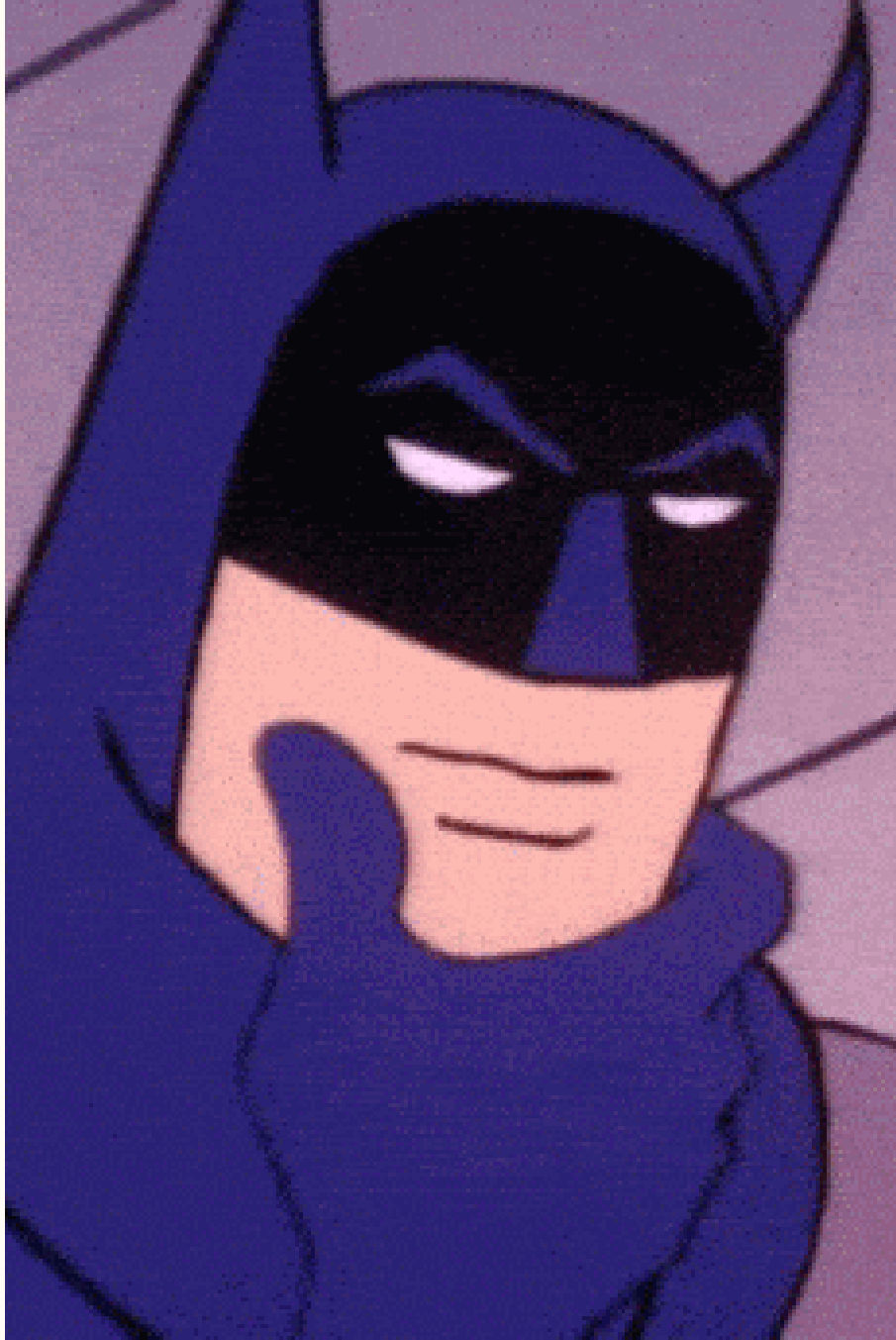
Report and S-Box LUT

Ali Mohamed

Schematic and MUX

Youssef Ahmed

Presentation and Shifter

Thank You!

Any questions?